DevOps Final Capstone • Session 21

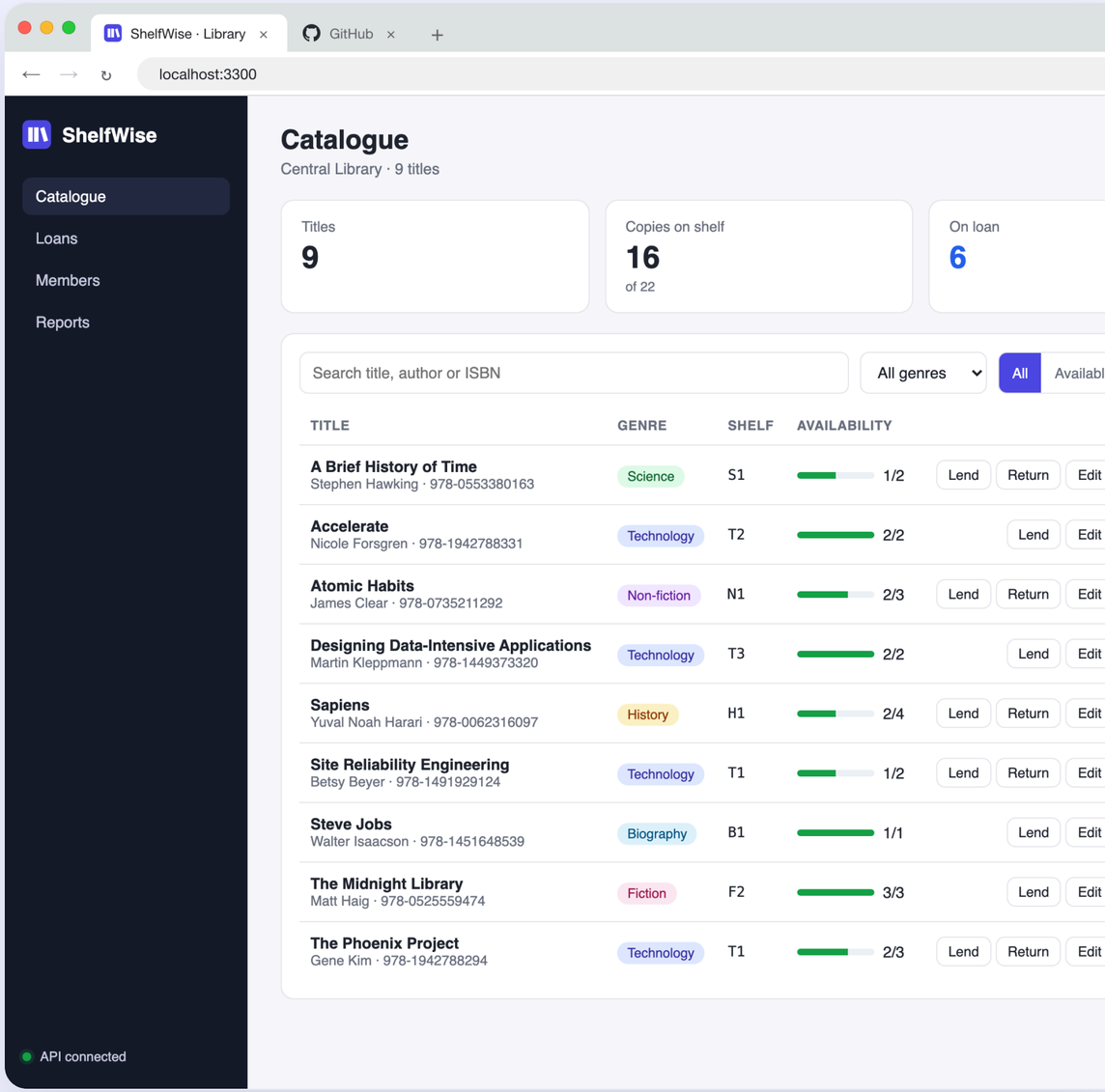
ShelfWise

Library management, from a developer laptop to a monitored Kubernetes cluster

Mohammed Abdurrahman · 24BCS10130

Scaler School of Technology · October 2026

github.com/abdurrahmaan11265/shelfwise-devops-capstone



The application

A catalogue, the copies on each shelf, and who has borrowed what

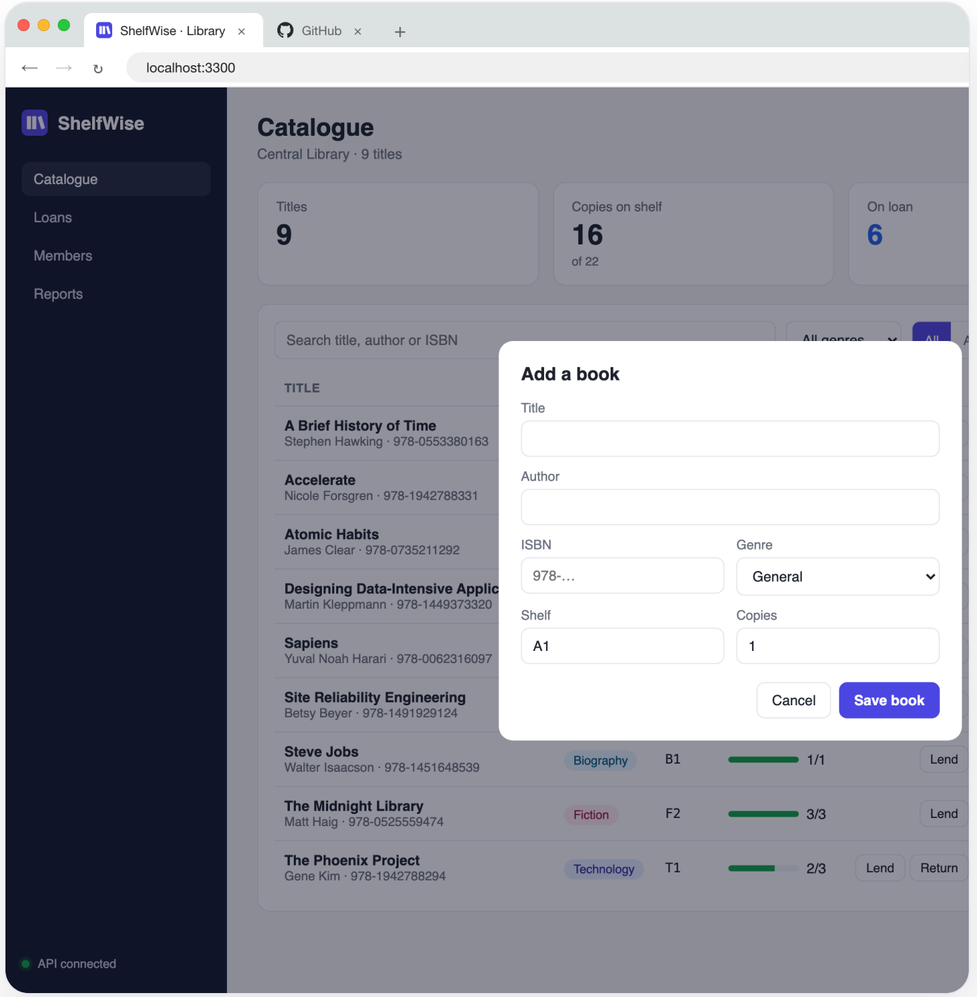
Librarians search and filter the catalogue, add and edit books, lend copies to borrowers, take them back, and see what is on loan or overdue at a glance.

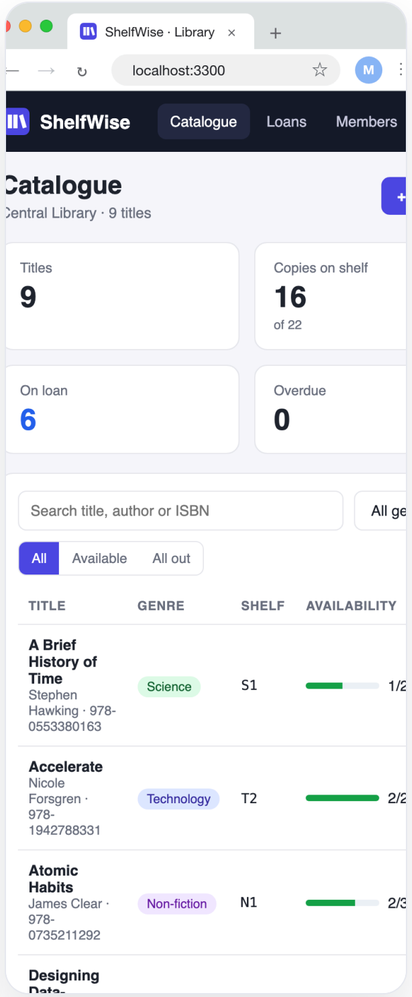
My own domain, built on the course architecture: React + Vite in front, FastAPI and PostgreSQL behind, SQLAlchemy models and Alembic migrations.

14
REST routes

2
Alembic migrations

3
services in Compose





The delivery path

Every commit travels the same road, and every step leaves evidence

Git / GitHub
20 descriptive commits, public repo

→

GitHub Actions
pytest · frontend build · Docker build · Trivy gate · push to GHCR · Helm deploy + smoke test

→

Terraform
AWS VPC, subnets, NAT, IAM, EKS 1.36 + node group

→

Helm → Kubernetes
Deployments, ClusterIP Services, Ingress, HPA, probes

→

Prometheus + Grafana
/metrics scraped, golden-signals dashboard

GRADING MODULES COVERED IN THIS DECK

M1 Application · 10 | **M2** Testing · 10 | **M3** Git · 5 | **M4** Docker · 10 | **M5** CI/CD · 15 | **M6** Trivy · 5 | **M7** Terraform · 15 | **M8** Kubernetes + Helm · 15 | **M9** Observability · 10 | **M10** Docs + demo · 5

Image tag = commit SHA

Every running pod traces back to an exact commit, and the UI sidebar shows it.

Push only after the Trivy gate

A vulnerable image is never published to the registry.

Deploy job on a throw-away kind cluster

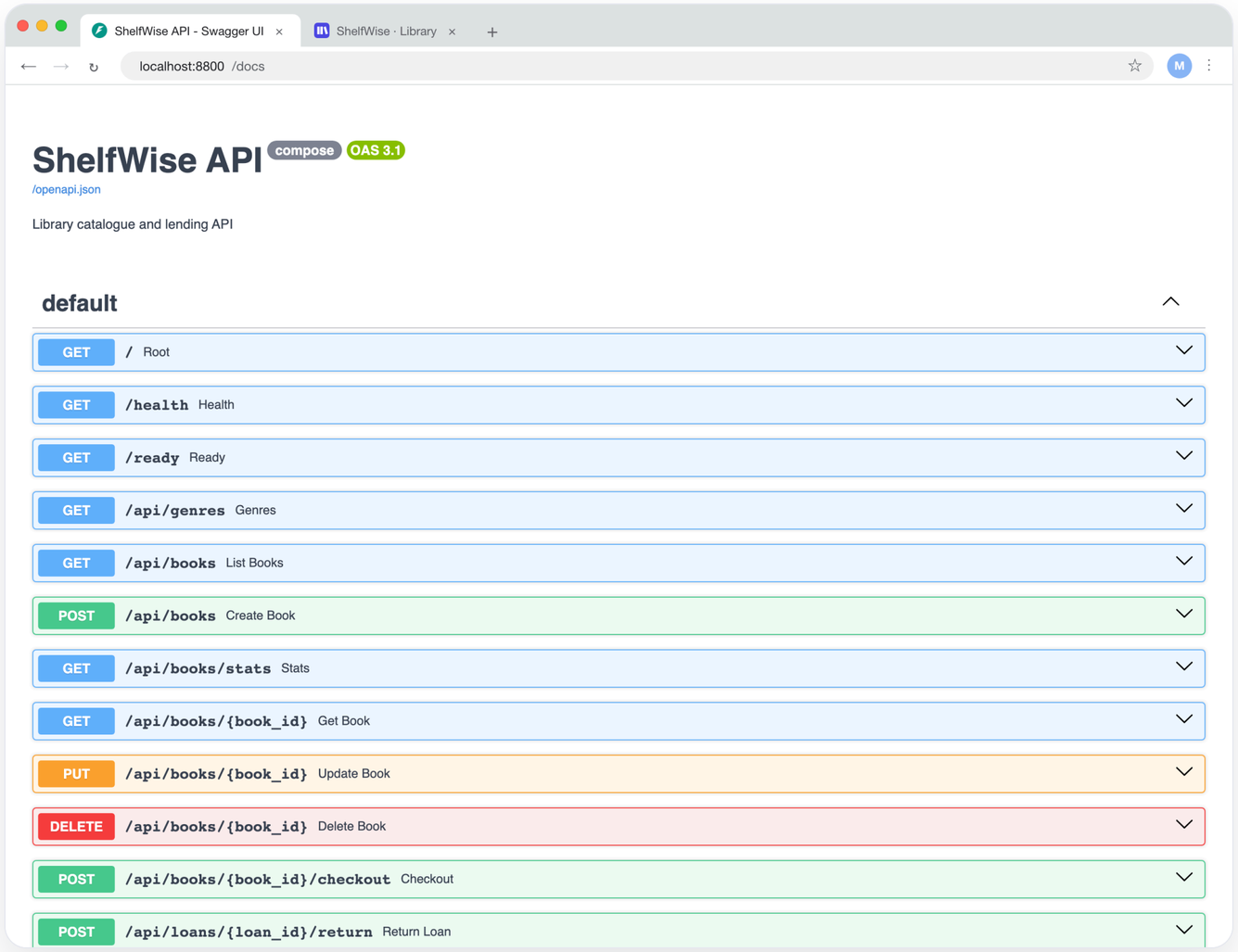
Each commit proves the chart installs and the app answers through the Ingress.

M1 Application · 10 pts

FastAPI backend, PostgreSQL via Alembic, React frontend

ROUTE	PURPOSE
`GET /health · GET /ready`	liveness without the DB, readiness with it
`GET /api/books`	list, search and filter the catalogue
`POST · PUT · DELETE /api/books`	create, edit, remove a book
`POST …/checkout · …/return`	lend a copy, take it back
`GET /api/books/stats · /api/loans`	dashboard numbers, what is on loan
`GET /metrics · GET /api/info`	Prometheus metrics, running build SHA

Copy-count rules live as database check constraints, so the invariant holds even if a bug slips past the Python validation.



Swagger UI at /docs, generated from the FastAPI schemas

M2 Testing · 10 pts

Tests are the first gate: nothing is built until they pass

12 **Test cases over 8 endpoints**

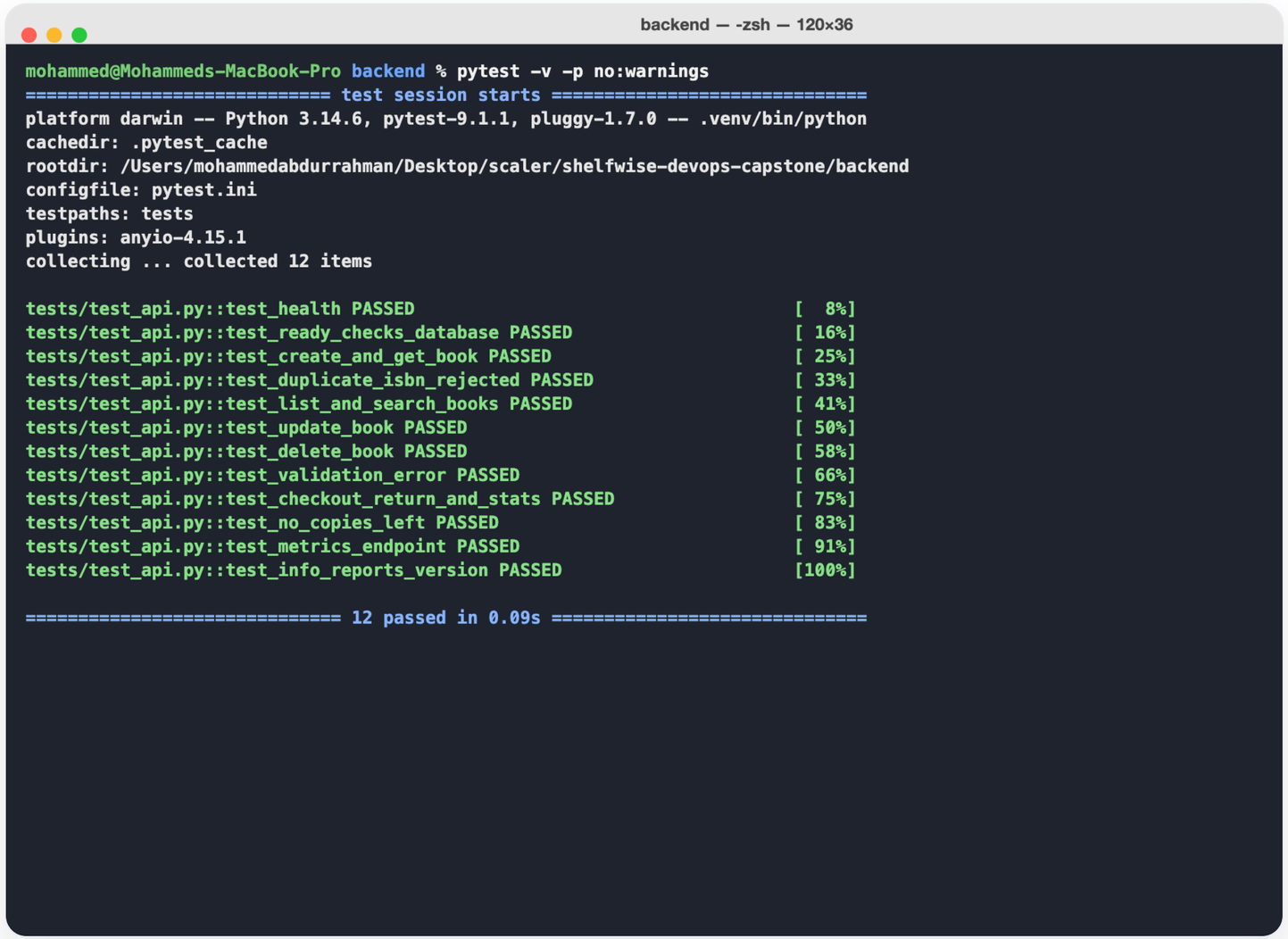
CRUD, validation (409 duplicate ISBN, 422 bad input), lending rules, health, metrics.

DB **Never the production database**

conftest.py overrides the FastAPI get_db dependency with a fresh in-memory SQLite per test.

CI **Runs first in the pipeline**

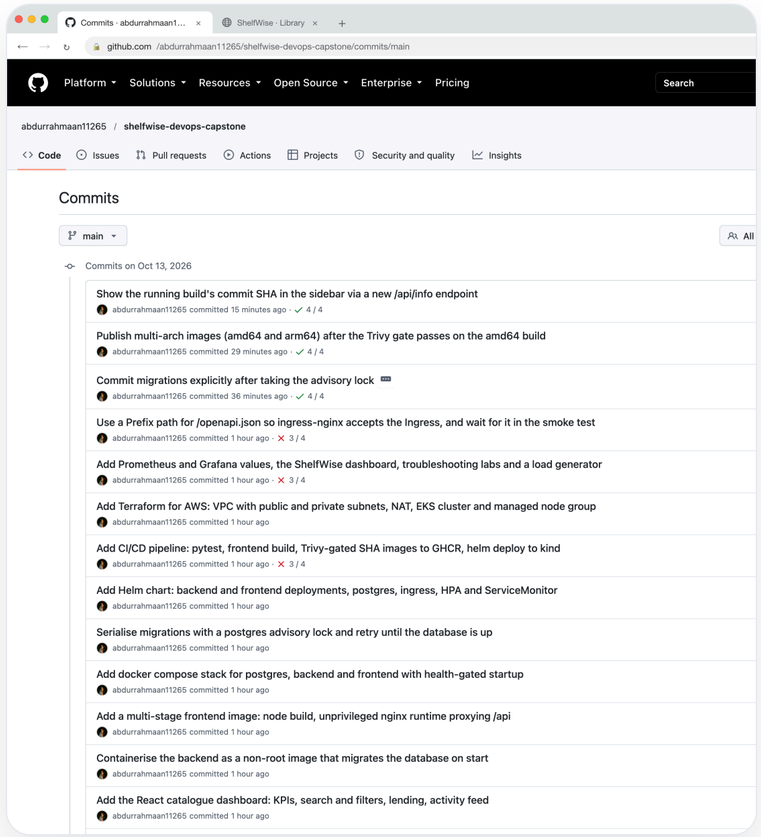
A failing test stops the build job, so a broken image is never promoted.



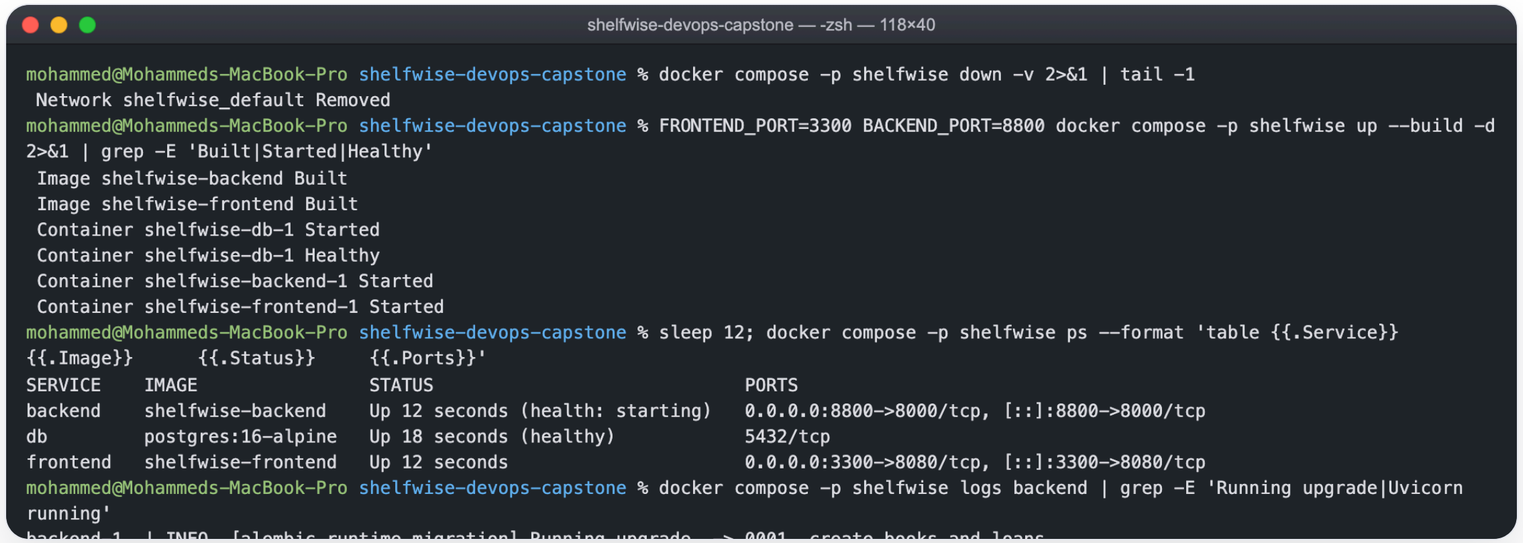
pytest -v, 12 passed

M3 Git · 5 pts **M4** Docker · 10 pts

One commit per step, and the whole stack in one command



20 commits, each a readable step



docker compose up --build: postgres, backend, frontend on :3000

Backend image

python:3.12-slim, OS packages upgraded, runs as uid 10001, HEALTHCHECK on /health

Frontend image

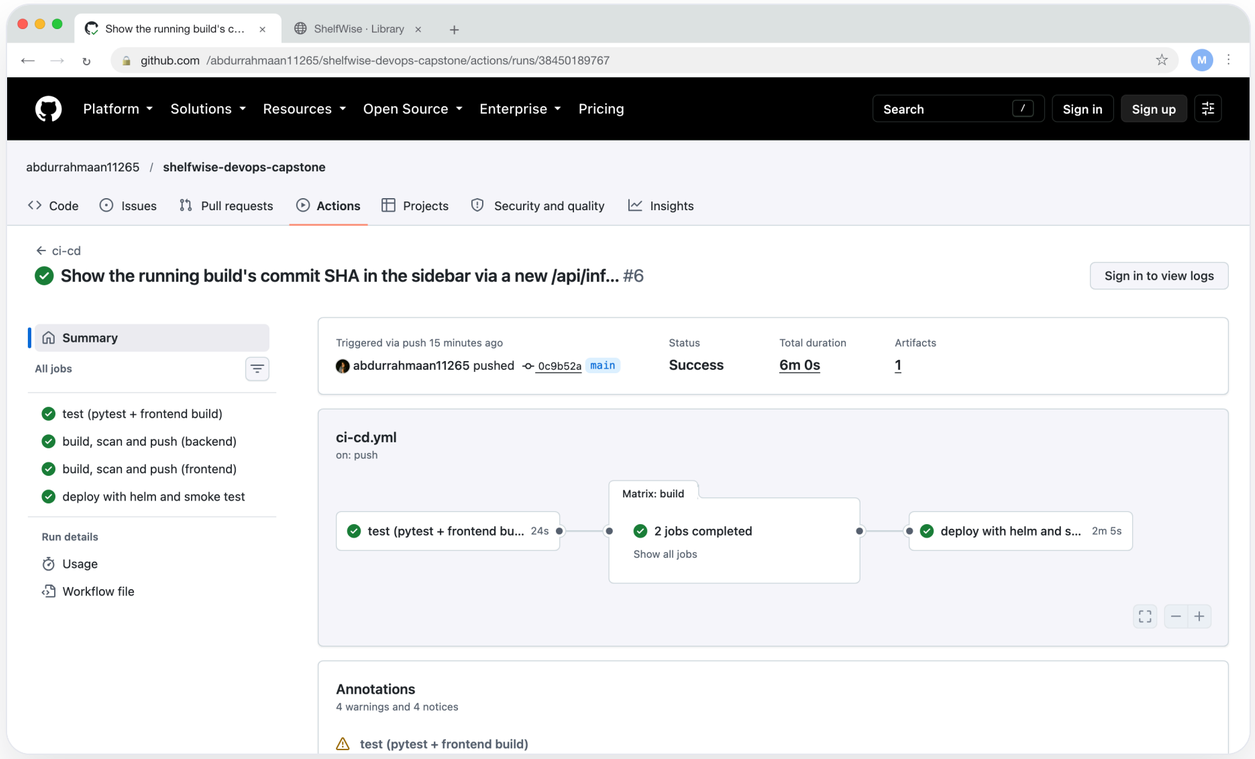
Multi-stage: node:22 builds, nginx-unprivileged serves dist/ as uid 101

.gitignore

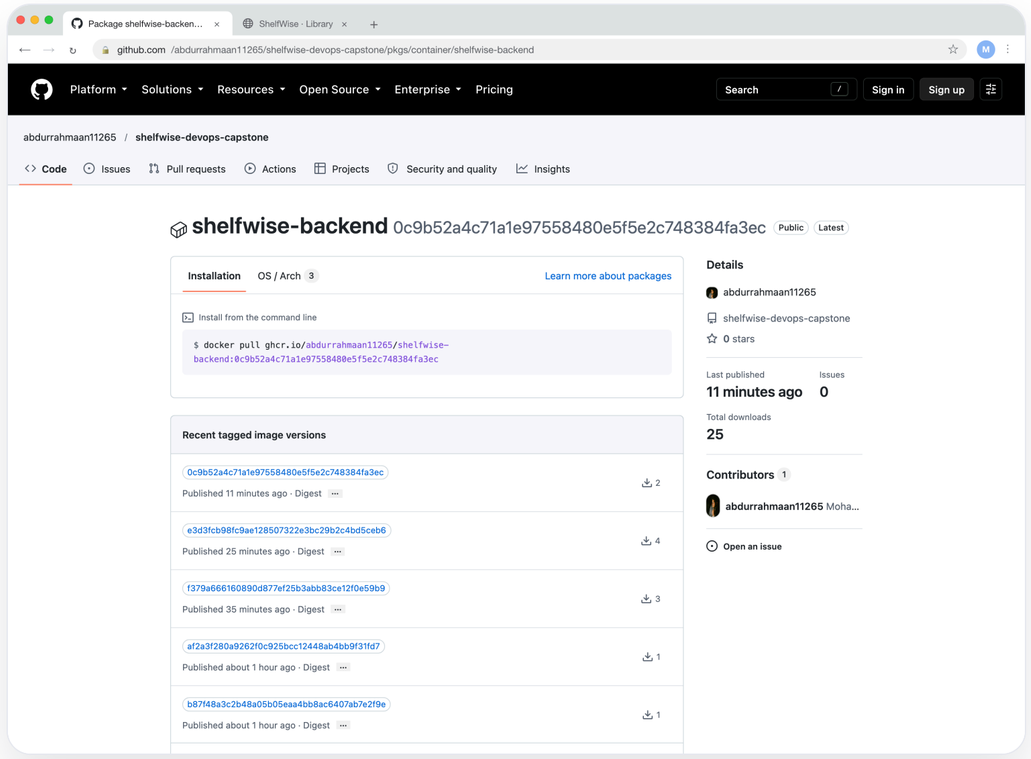
.env, __pycache__, node_modules, .venv, tfstate and tfvars never committed

M5 CI/CD · 15 pts

test → build + scan + push → deploy, on every push to main



A green run: test, build (backend and frontend in parallel), deploy



GHCR packages tagged with the commit SHA, never latest

Frontend built in CI

npm ci and npm run build in the test job; a failed build stops the pipeline.

Multi-arch images

amd64 for EKS nodes, arm64 for Apple silicon and Graviton, from one workflow.

Deploy + smoke test

helm upgrade --install on a fresh kind cluster, then create a book through the Ingress and read it back.

M6 DevSecOps · 5 pts

Trivy scans both images, and the push waits for the result

```
trivy image --severity HIGH,CRITICAL \
  --ignore-unfixed --exit-code 1 <image>
```

1. **What it scans**

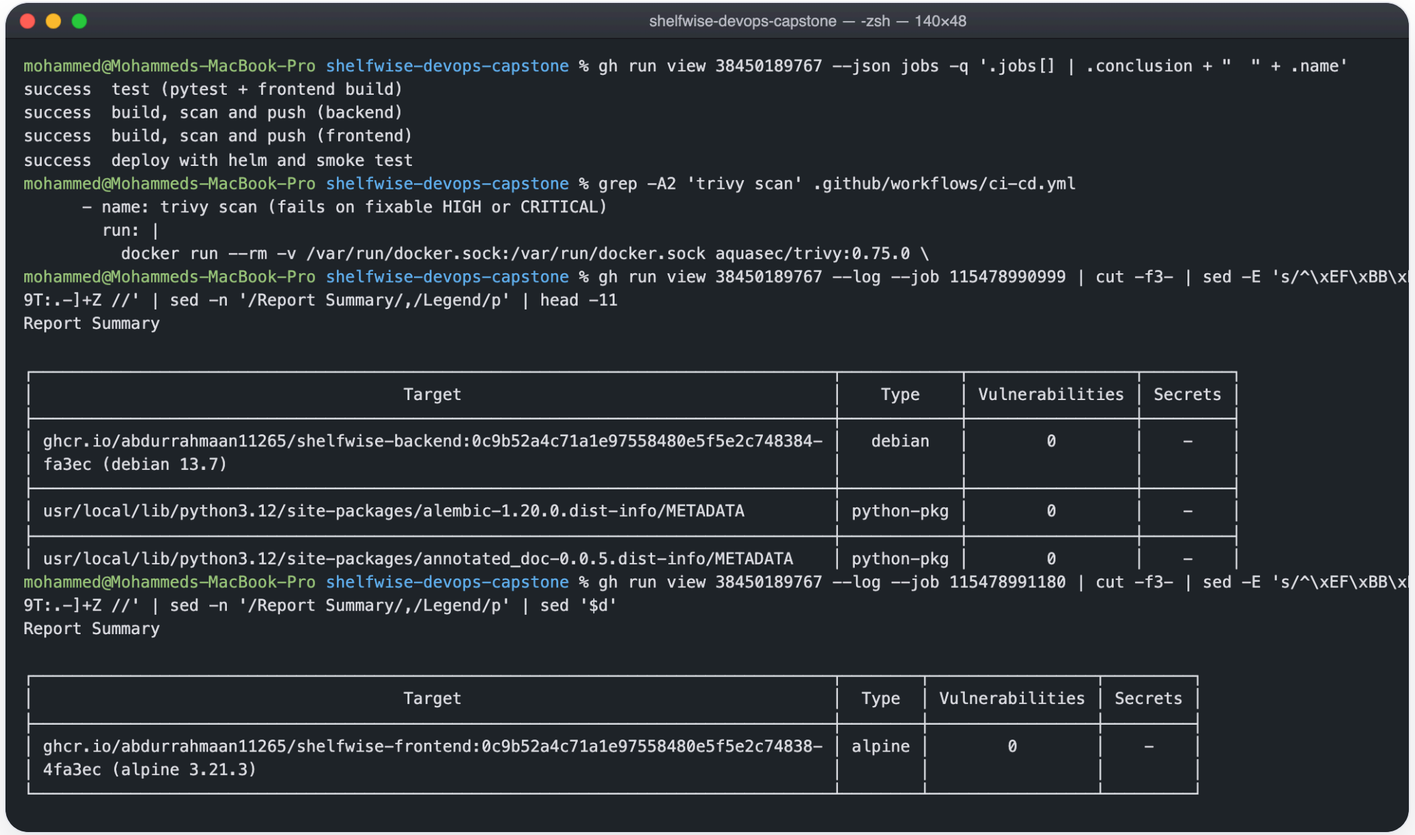
   OS packages of each image (Debian for the backend, Alpine for the frontend) and the Python wheels, against vulnerability databases.

2. **What the result means**

   Both images are clean: zero fixable HIGH or CRITICAL findings at build time. Base images are upgraded during the build, and Python dependencies are pinned to current releases.

3. **Why it is a gate, not a report**

   exit-code 1 fails the job, and the push step comes after the scan, so a vulnerable image is never published.



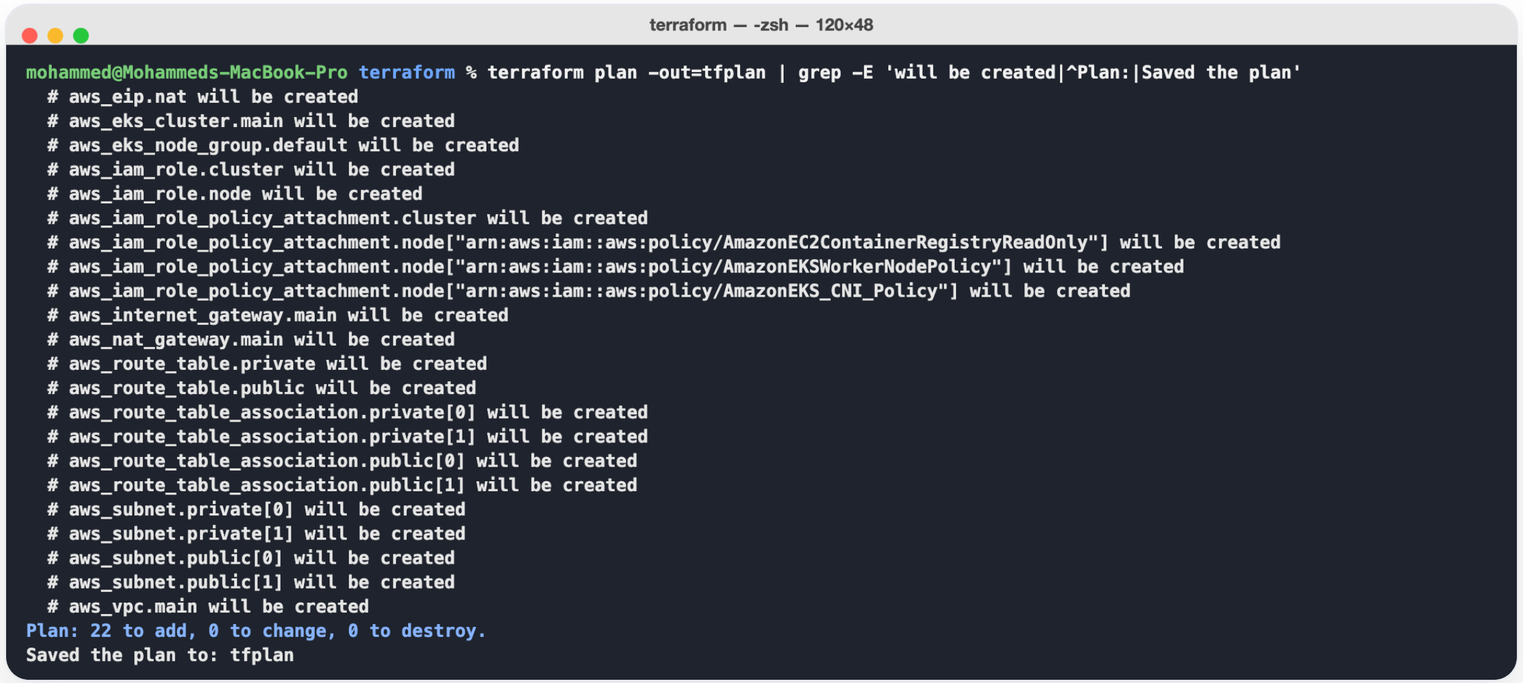
Trivy output from the pipeline run

M7 Terraform · 15 pts

22 resources in HCL: a VPC across two AZs and an EKS cluster

FILE	CREATES
`vpc.tf`	VPC 10.40.0.0/16, 2 public + 2 private subnets, IGW, NAT gateway, route tables
`eks.tf`	cluster and node IAM roles, EKS 1.36 control plane, managed node group of 2 × t3.medium
`variables.tf`	region ap-south-1, CIDRs, versions, node sizes
`tfvars.example`	committed; the real tfvars and all credentials are not

Subnets carry the kubernetes.io/role/elb tags so the load balancer controller can place load balancers for the Ingress.



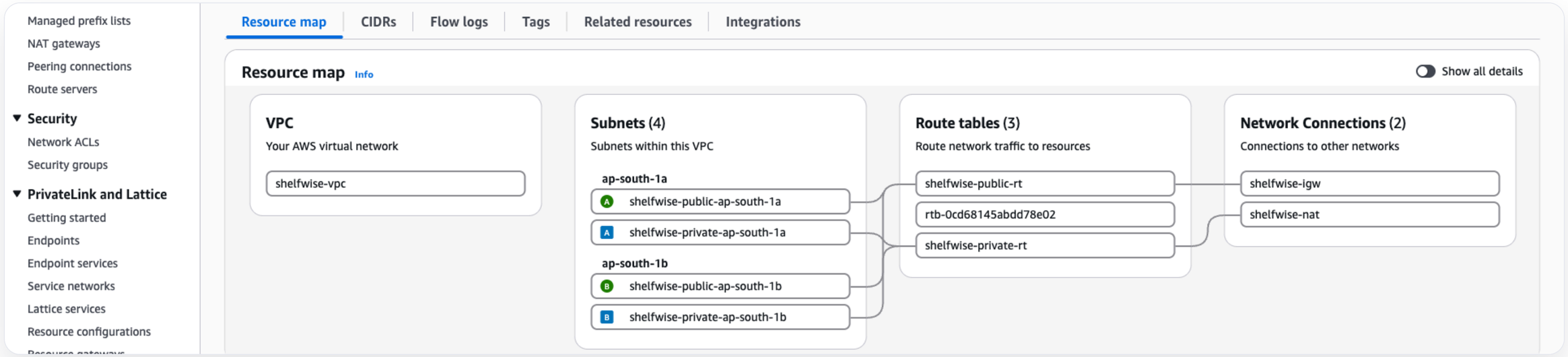
terraform plan after init, fmt -check and validate: 22 to add, 0 to change, 0 to destroy

4
subnets across ap-south-1a and 1b

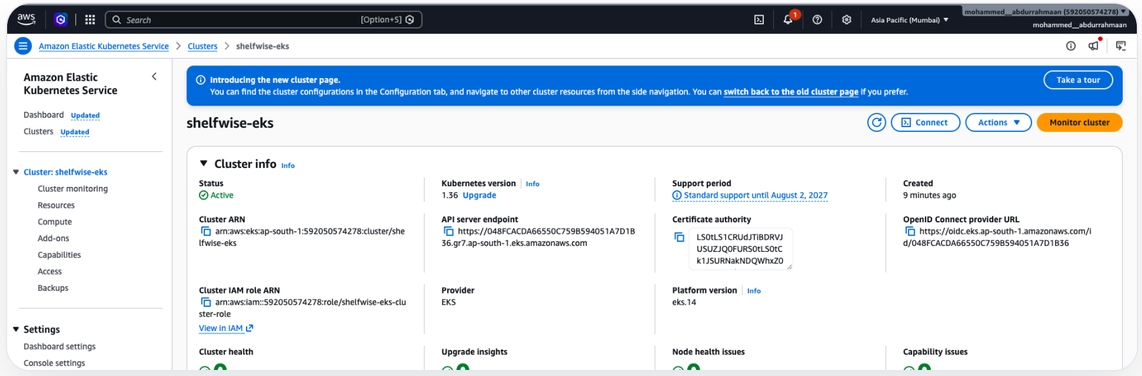
1
NAT gateway for the private nodes

0
credentials in the repo or its history

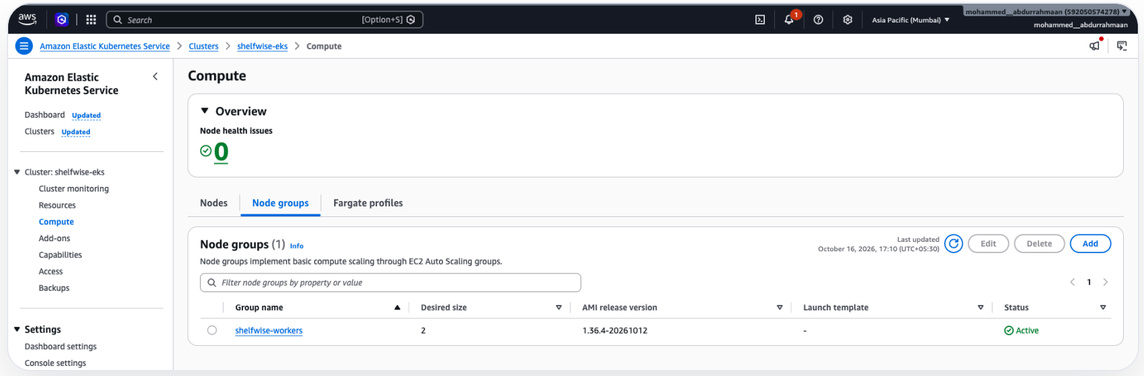
The console shows what the code describes: VPC, cluster, node group



VPC resource map: four subnets in two AZs, public and private route tables, Internet gateway and NAT gateway



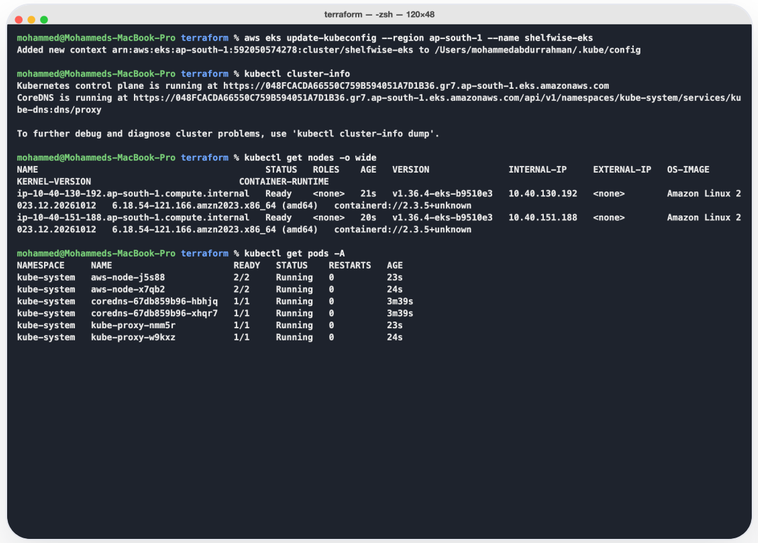
EKS: shelfwise-eks Active, Kubernetes 1.36, standard support



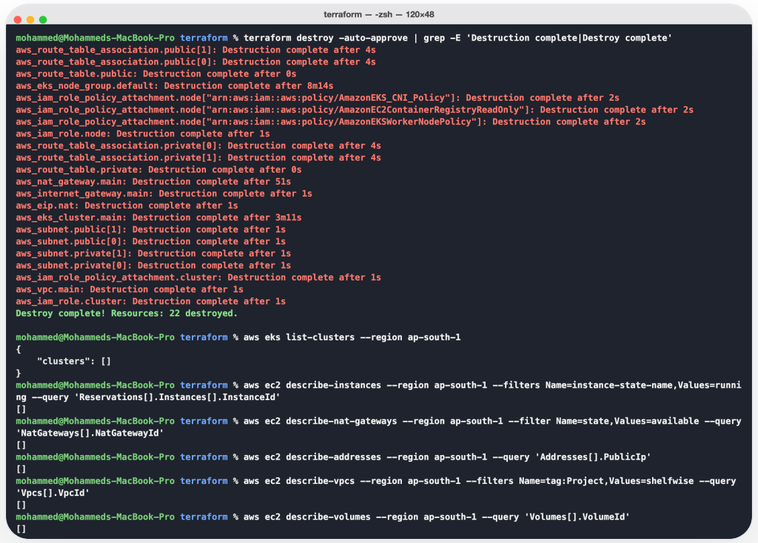
Compute: node group shelfwise-workers, desired size 2, Active

M7 Terraform · the real run, start to finish

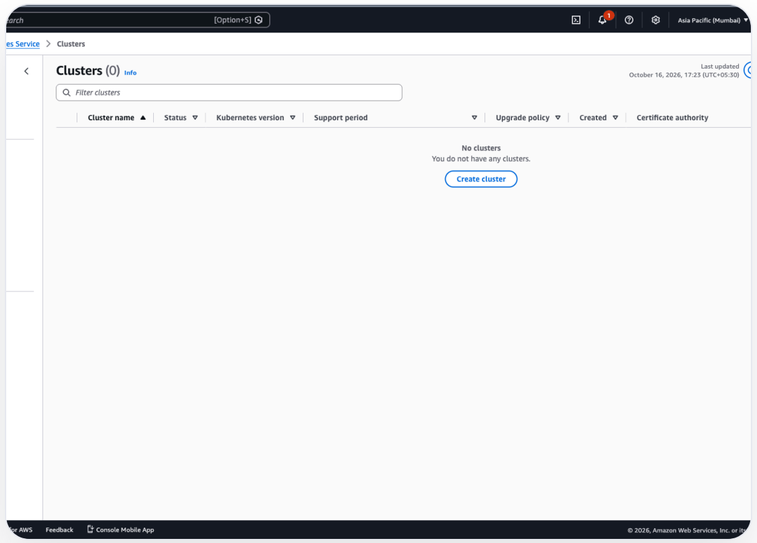
Applied, reached with kubectl, destroyed: 23 minutes, under a dollar



update-kubeconfig, then both t3.medium nodes Ready and the system pods Running



terraform destroy: 22 destroyed, then every CLI check returns empty



EKS console afterwards: no clusters

22
resources created, then 22 destroyed

8m 38s
to create the EKS control plane

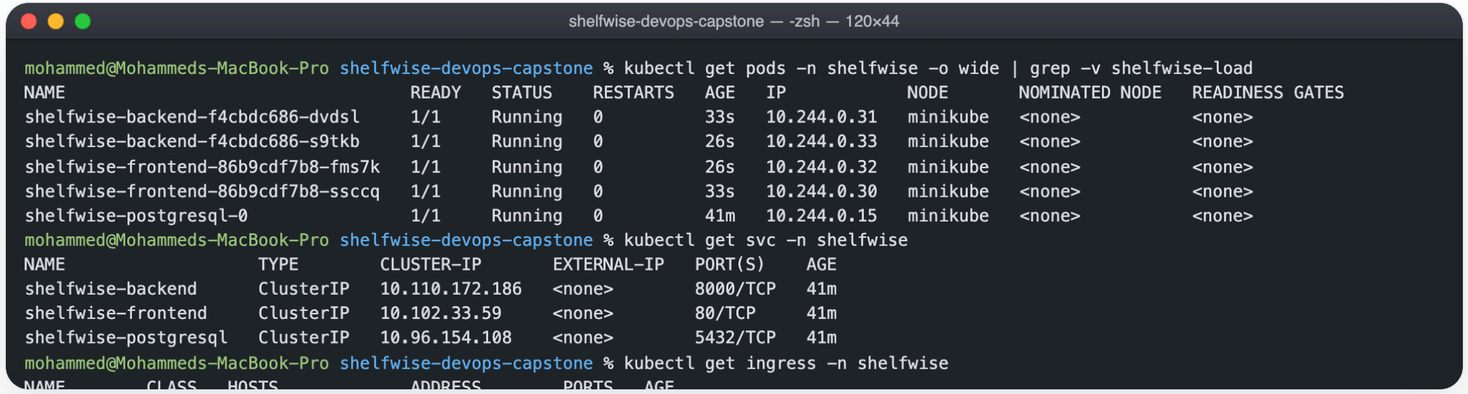
23 min
from first apply call to destroy complete

< $1
with a US$1 AWS Budget as the guardrail

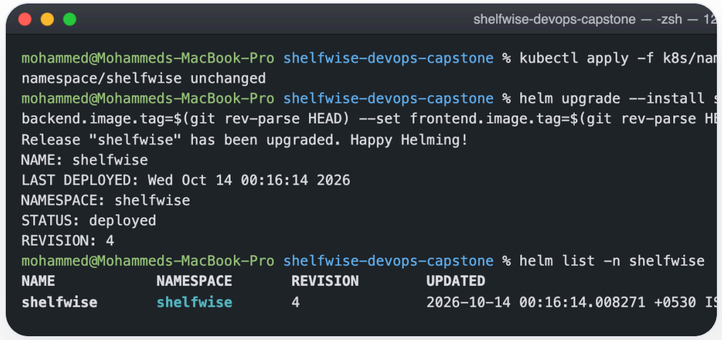
M8 Kubernetes + Helm · 15 pts

One chart creates the whole application, and the Ingress fronts it

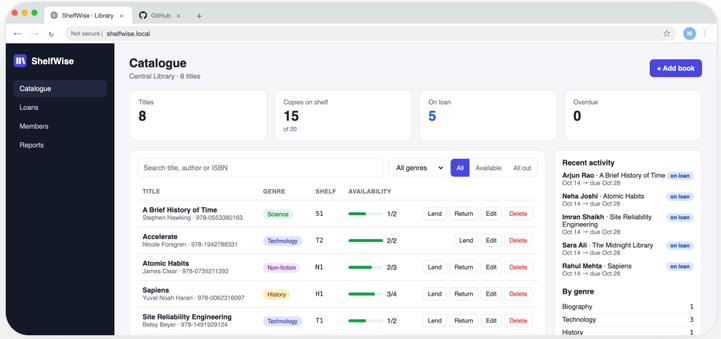
OBJECT	DETAILS
Deployments	backend and frontend, 2 replicas each, startup, liveness and readiness probes, non-root
Services	ClusterIP for backend, frontend and PostgreSQL
Ingress	nginx, shelfwise.local: /api and /docs → backend, / → frontend
HPA	backend 2 → 6 pods at 60% CPU, frontend 2 → 4
StatefulSet	PostgreSQL with a 1Gi PVC; password generated by Helm, kept across upgrades



kubectl get pods, svc, ingress: all Running, ClusterIP services, nginx Ingress



helm upgrade --install and helm list



The app through shelfwise.local

M8 Autoscaling

Under load the backend scales from 2 to 6 pods on its own

1 **Resource requests set**

HPA needs CPU requests on the pods and metrics-server in the cluster; both are in place.

2 **Load generated**

scripts/load-test.sh hammers the API while kubectl get hpa -w watches.

3 **Scaled to the maximum**

CPU hit 436% of target; replicas went 2 → 6, then back down when the load stopped.

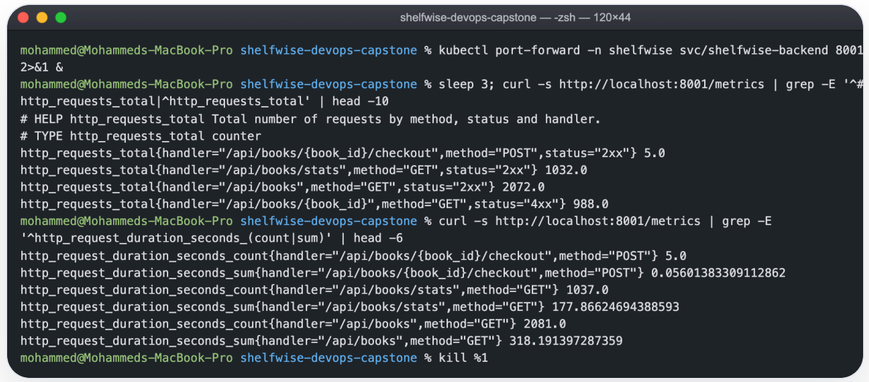
shelfwise-devops-capstone — -zsh — 120×44

```
mohammed@Mohammeds-MacBook-Pro shelfwise-devops-capstone % kubectl get hpa -n shelfwise
NAME                 REFERENCE                     TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
shelfwise-backend    Deployment/shelfwise-backend    cpu: 436%/60%   2         6         6          43m
shelfwise-frontend   Deployment/shelfwise-frontend   cpu: 2%/70%     2         4         2          43m
mohammed@Mohammeds-MacBook-Pro shelfwise-devops-capstone % kubectl top pods -n shelfwise
NAME                                 CPU(cores)   MEMORY(bytes)
shelfwise-backend-f4cbdc686-dvdsl    449m         73Mi
shelfwise-backend-f4cbdc686-s9tkb    424m         72Mi
shelfwise-frontend-86b9cdf7b8-fms7k  1m           11Mi
shelfwise-frontend-86b9cdf7b8-ssccq  1m           11Mi
shelfwise-load                       336m         16Mi
shelfwise-postgresql-0               170m         63Mi
mohammed@Mohammeds-MacBook-Pro shelfwise-devops-capstone % kubectl get pods -n shelfwise -l app.kubernetes.io/component=backend
NAME                                 READY   STATUS    RESTARTS   AGE
shelfwise-backend-f4cbdc686-2x74t    1/1     Running   0          23s
shelfwise-backend-f4cbdc686-997wx    1/1     Running   0          38s
shelfwise-backend-f4cbdc686-dfs5q    1/1     Running   0          23s
shelfwise-backend-f4cbdc686-dvdsl    1/1     Running   0          2m53s
shelfwise-backend-f4cbdc686-hhtmg    1/1     Running   0          38s
shelfwise-backend-f4cbdc686-s9tkb    1/1     Running   0          2m46s
mohammed@Mohammeds-MacBook-Pro shelfwise-devops-capstone % kubectl describe hpa shelfwise-backend -n shelfwise | grep -A6 '^Events'
Events:
  Type     Reason                        Age                From                       Message
  ----     ------                        ----               ----                       -------
  Normal   SuccessfulRescale             43m                horizontal-pod-autoscaler  New size: 2; reason: Current number of repli
below Spec.MinReplicas
  Normal   SuccessfulRescale             24m                horizontal-pod-autoscaler  New size: 2; reason: All metrics below targe
  Warning  FailedGetResourceMetric       10m (x22 over 37m)  horizontal-pod-autoscaler  failed to get cpu utilization: did not recei
metrics for targeted pods (pods might be unready)
  Warning  FailedComputeMetricsReplicas  10m (x22 over 37m)  horizontal-pod-autoscaler  invalid metrics (1 invalid out of 1), first
is: failed to get cpu resource metric value: failed to get cpu utilization: did not receive metrics for targeted pods (pods might be
unready)
```

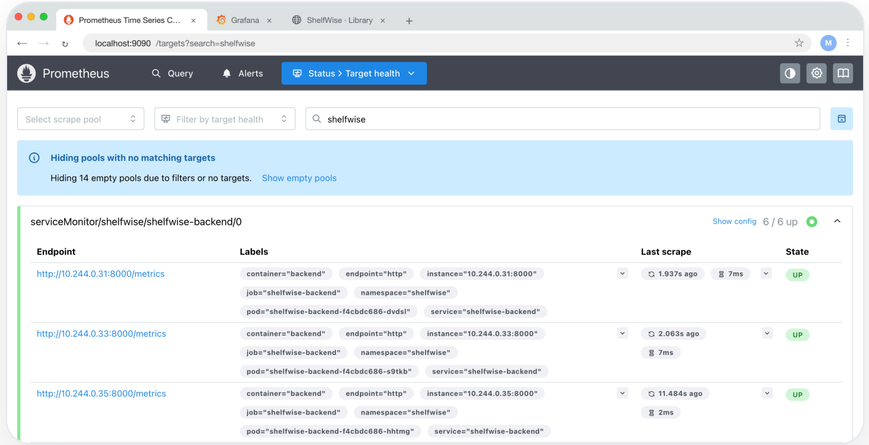
kubectl get hpa -n shelfwise during the load test

M9 Observability · 10 pts

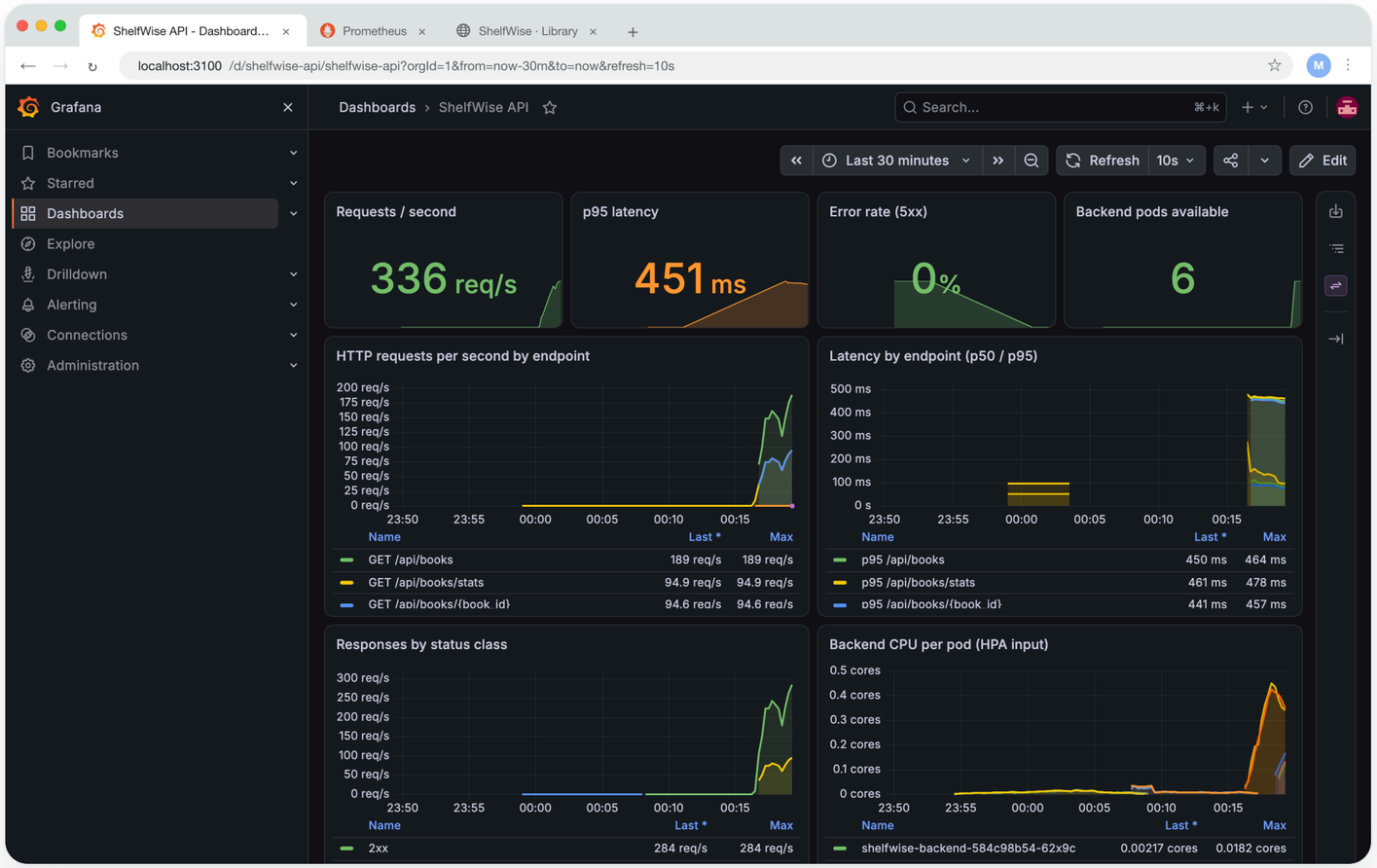
Metrics exposed, scraped by Prometheus, shown in Grafana



curl /metrics: request counts and latency histograms



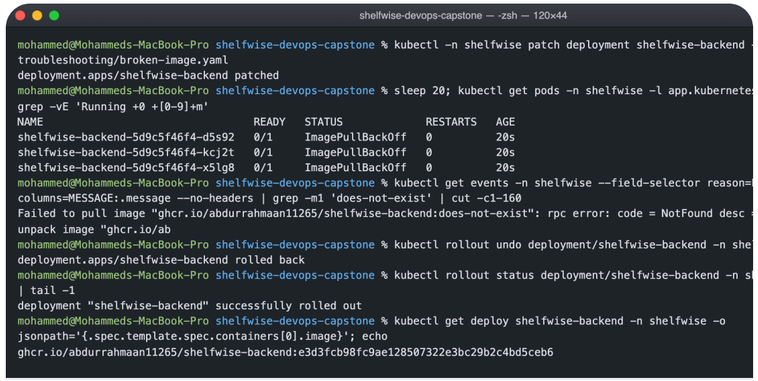
Prometheus targets: every backend pod UP via the ServiceMonitor



ShelfWise API dashboard: requests/s, p95 latency, 5xx rate, pods, per-endpoint panels, CPU per pod

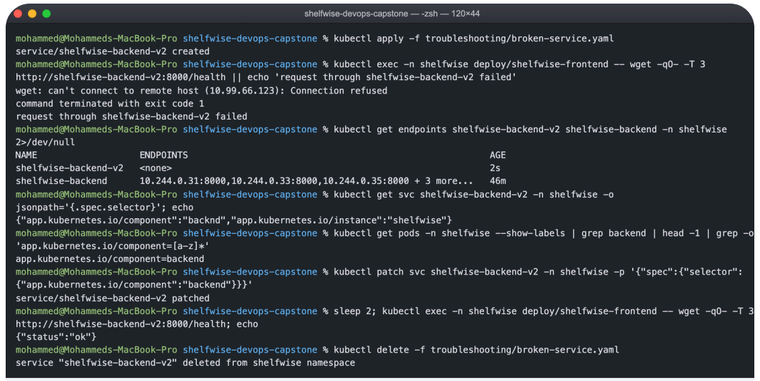
Troubleshooting labs

Three deliberate breakages, each diagnosed and fixed on the cluster



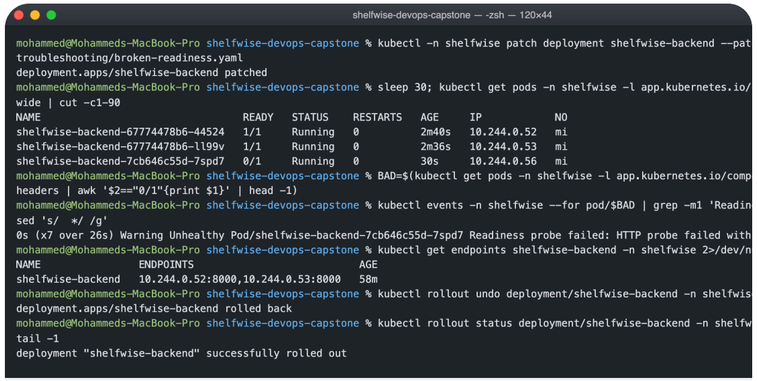
Broken image tag

Symptom new pods ImagePullBackOff, old pods keep serving.
Clue the Failed event names a tag that was never pushed.
Fix kubectl rollout undo.



Broken Service selector

Symptom ClusterIP exists, connections refused.
Clue no endpoints; one-letter typo versus the pod labels.
Fix patch the selector.

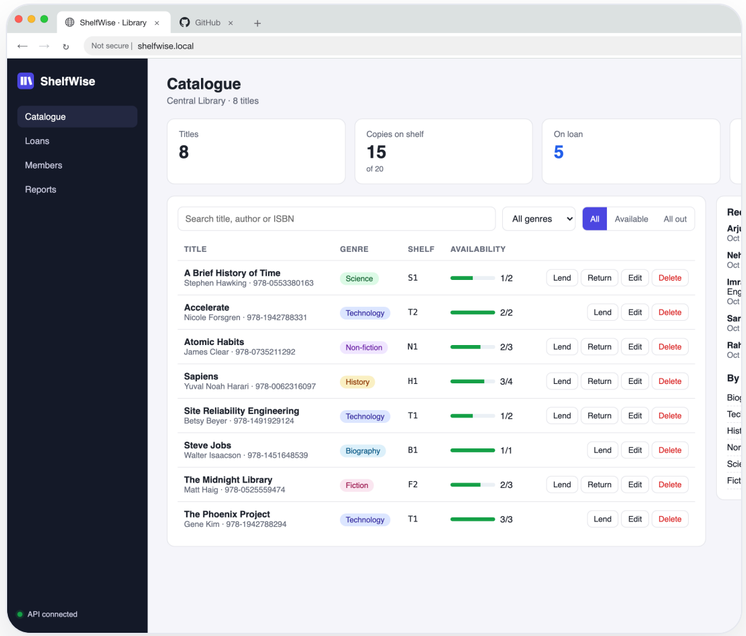


Broken readiness probe

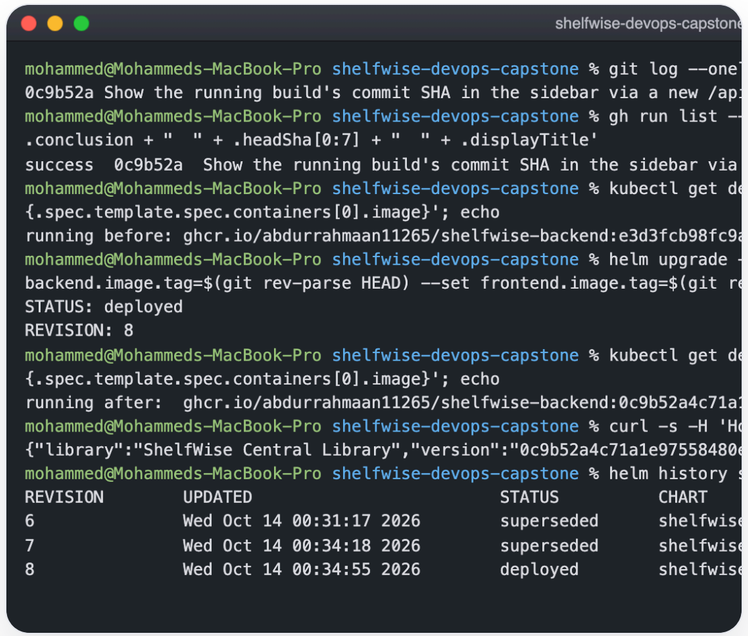
Symptom pod Running but 0/1, rollout stalls.
Clue probe 404 on /readyz; pod IP missing from endpoints.
Fix kubectl rollout undo.

M10 Demo · commit to deployment

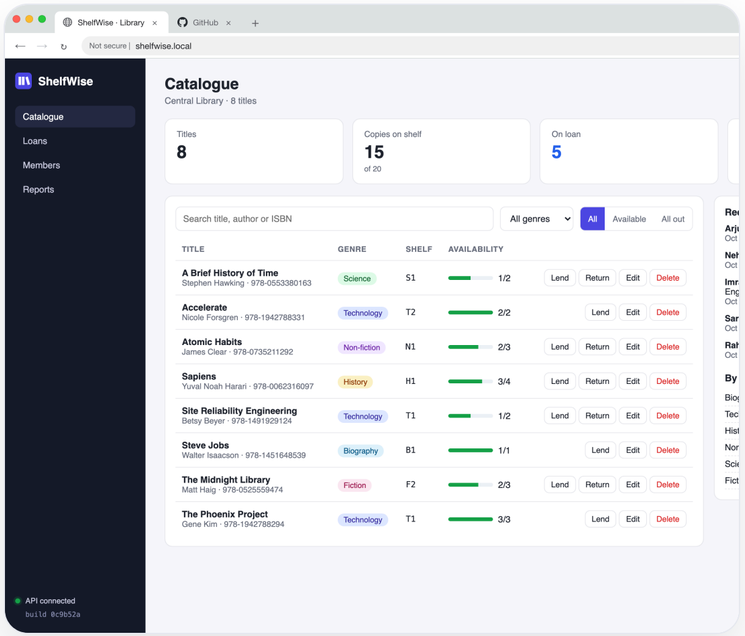
Change the code, push, and watch the running build update



1 · Before. The cluster runs the previous build; the sidebar has no build line.



2 · Pipeline and deploy. Commit 0c9b52a is tested, scanned, pushed as its SHA, then helm upgrade rolls the pods.



3 · After. The same page through the Ingress now shows build 0c9b52a.

Live demo plan

Edit the sidebar text, commit, push. Show the Actions run go green, then the new SHA in the GHCR package and in the UI.

If the network misbehaves

These three screenshots and the run history on GitHub stand in for the live run.

Two bugs the pipeline caught

An Ingress path rejected by nginx validation, and migrations silently rolled back inside an open transaction. Both visible in the run history.

Design decisions

Why each piece is the way it is

/health does not touch the DB, /ready does

A database outage should stop traffic through readiness, not restart healthy pods in a loop through liveness.

Migrations at container start, under an advisory lock

No separate job to orchestrate, and safe when N replicas start together.

Kubernetes 1.36, not 1.31

1.31 is now in EKS extended support, billed at six times the control-plane rate. Checked before the run.

Private subnets for the nodes

Workers reach the internet through the NAT gateway; nothing is exposed except through the Ingress.

PostgreSQL in the cluster for the demo

Simple classroom setup. In production a managed database such as RDS brings backups, Multi-AZ and patching.

Minikube for the app, EKS for the proof

The cluster was created, verified and destroyed in one 23-minute session; the same Helm command applies after update-kubeconfig.

WHAT I WOULD ADD NEXT

Remote Terraform state

S3 backend with DynamoDB locking, so a team can share the state safely.

Managed PostgreSQL

Amazon RDS for backups, Multi-AZ failover and automatic patching.

TLS on the Ingress

cert-manager with Let's Encrypt, HTTPS only.

GitOps deploys

Argo CD watching the chart, instead of the pipeline pushing with Helm.

Thank you

Questions?

Every module has its evidence linked from the README's grading table, and the repository is public.

`github.com/abdurrahmaan11265/shelfwise-devops-capstone`

README → 17. Grading evidence

Mohammed Abdurrahman · 24BCS10130

